

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN



BÁO CÁO THỰC HÀNH
MÔN HỌC: TÍNH TÍNH SONG SONG

Giảng viên hướng dẫn: TS Nguyễn Thành Nam
Nhóm học viên thực hiện: Phạm Minh Thùy - MSHV: 20261051M
Trần Thị Khánh Huyền - MSHV: 20252875M

Hà Nội, Năm 2026

Mục lục

Giới thiệu về báo cáo	2
1 Bài tập 1: Tích vô hướng hai Vector	3
1.1 Mô tả bài toán	3
1.2 Phân tích mã nguồn	3
1.3 Kết quả thực nghiệm	4
2 Bài tập 2: Giải hệ phương trình tuyến tính	7
2.1 Lý thuyết phương pháp khử Gauss	7
2.2 Phân tích song song hóa bằng OpenMP	7
2.2.1 Ví dụ minh họa khử Gauss từng bước hệ phương trình 4×4	9
2.3 Kết quả thực nghiệm và đánh giá hiệu năng Gauss	11
2.3.1 Kiểm chứng tính đúng đắn	11
2.3.2 Cấu hình môi trường thực nghiệm	12
2.3.3 Thực nghiệm hiệu năng với dữ liệu lớn	12
2.3.4 Biểu đồ kết quả đo lường hiệu năng	13
2.3.5 Phân tích và giải thích hiện tượng hiệu năng	14
2.3.6 Hướng tối ưu hóa và cải tiến	16
2.3.7 Tối ưu hóa nâng cao: Thuật toán chia khối (Tiling/Blocked LU) . .	19
2.4 Phương pháp lặp Jacobi (Khung đề mục hoàn thiện)	23
Kết luận	24
Tài liệu tham khảo	25

Giới thiệu về báo cáo

Môn học **Tính toán song song** cung cấp các kiến thức cơ bản và nâng cao về lập trình đa xử lý trên hệ thống máy tính hiện đại. Trong báo cáo này, nhóm thực hiện lập trình và thử nghiệm các bài toán số học cơ bản bằng công cụ **OpenMP** (Open Multi-Processing) - một API hỗ trợ lập trình song song bộ nhớ chia sẻ phổ biến và mạnh mẽ trên C/C++ và Fortran.

Nội dung báo cáo tập trung vào hai bài tập thực hành chính:

1. **Bài tập 1: Tính tích vô hướng của hai vector:** Thực hiện song song hóa vòng lặp tính toán tích vô hướng trên vector, thử nghiệm hiệu năng và tính đúng đắn với các kích thước dữ liệu khác nhau.
2. **Bài tập 2: Giải hệ phương trình tuyến tính:** Sử dụng hai phương pháp phổ biến là phương pháp khử Gauss và phương pháp lặp Jacobi. Báo cáo này sẽ trình bày chi tiết về phần thuật toán và lập trình song song của phương pháp khử Gauss. Phần phương pháp Jacobi sẽ do thành viên khác trong nhóm phụ trách hoàn thiện.

Báo cáo bao gồm phân tích lý thuyết thuật toán, giải thích chi tiết từng đoạn mã nguồn C++ có tích hợp OpenMP và đưa ra các kết quả thử nghiệm thực tế từ chương trình.

Chương 1

Bài tập 1: Tích vô hướng hai Vector

1.1 Mô tả bài toán

Cho hai vector số thực $A = (a_0, a_1, \dots, a_{n-1})$ và $B = (b_0, b_1, \dots, b_{n-1})$ cùng kích thước n .

Tích vô hướng hai vector là phép toán cho kết quả là một số thực $S = A \cdot B$ được tính theo công thức:

$$S = \sum_{i=0}^{n-1} a_i \times b_i \quad (1.1)$$

Để tính toán tích vô hướng, ta thực hiện cộng dồn kết quả nhân các cặp phần tử tương ứng $a_i \times b_i$ vào một biến vô hướng duy nhất S . Trong lập trình song song, phép tính tích lũy này đòi hỏi cơ chế kiểm soát tranh chấp dữ liệu (race condition) trên biến chia sẻ S .

1.2 Phân tích mã nguồn

Dưới đây là mã C++ tính tích vô hướng hai vector bằng OpenMP:

```
1 double dotProduct(const vector<double>& a, const vector<double>& b) {
2     int n = static_cast<int>(a.size());
3     double sum = 0.0;
4
5     #pragma omp parallel for reduction(+:sum)
6     for (int i = 0; i < n; i++) {
7         sum += a[i] * b[i];
8     }
9     return sum;
10 }
```

Code 1.1: Thuật toán tích vô hướng song song sử dụng reduction

Giải thích hoạt động của mã nguồn:

- Dòng 2: `int n = static_cast<int>(a.size());`

Lấy kích thước của vector đầu vào. Hàm trả về kiểu `size_t` nên ta dùng `static_cast<int>` để chuyển về kiểu số nguyên `int` nhằm tương thích với biến chạy của chỉ thị vòng lặp song song trong OpenMP (yêu cầu biến chỉ số lặp thuộc kiểu nguyên).

- Dòng 3: `double sum = 0.0;`

Khai báo và khởi tạo biến tích lũy `sum` về 0. Đây là biến sẽ chứa kết quả tích vô hướng cuối cùng sau khi tất cả các luồng kết thúc.

• **Dòng 5: `#pragma omp parallel for reduction(+:sum)`**

Đây là chỉ thị song song hóa kèm cơ chế Reduction của OpenMP.

- `omp parallel for`: Yêu cầu trình biên dịch tạo ra một nhóm các luồng (thread team) và phân chia tự động các bước lặp của vòng lặp `for` cho các luồng xử lý đồng thời.
- `reduction(+:sum)`: Cơ chế Reduction giải quyết vấn đề tranh chấp dữ liệu khi nhiều luồng cùng cộng dồn vào một biến chia sẻ:
 1. OpenMP tạo ra bản sao cục bộ (private copy) của biến `sum` cho từng luồng, khởi tạo bằng giá trị trung hòa của phép cộng là 0.0.
 2. Mỗi luồng thực hiện cộng dồn tích $a_i \times b_i$ vào bản sao cục bộ của mình một cách độc lập.
 3. Sau khi vòng lặp kết thúc, OpenMP tự động cộng gộp (reduce) an toàn tất cả các bản sao cục bộ vào biến `sum` toàn cục để ra kết quả cuối cùng.

• **Dòng 6-8: `for (int i = 0; i < n; i++) { sum += a[i] * b[i]; }`**

Mỗi luồng nhân cặp phần tử $a_i \times b_i$ và cộng dồn vào biến cục bộ của mình. Cơ chế `reduction` đảm bảo tính toàn vẹn dữ liệu, loại bỏ tranh chấp dữ liệu (race condition) khi nhiều luồng tính toán đồng thời ghi vào một biến chia sẻ.

1.3 Kết quả thực nghiệm

Kiểm tra tính đúng đắn

Kết quả kiểm chứng tính đúng đắn của thuật toán tích vô hướng song song cho thấy kết quả thực thi hoàn toàn trùng khớp với giá trị tính toán lý thuyết:

Bảng 1.1: Kết quả kiểm tra tính đúng đắn của tích vô hướng

STT	Size (n)	Input (A và B)	Kết quả ($S = A \cdot B$)
1	3	$A = [1, 2, 3], B = [4, 5, 6]$	$S = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$
2	4	$A = [1, 2, 3, 4], B = [5, 6, 7, 8]$	$S = 5 + 12 + 21 + 32 = 70$
3	5	$A = [1..5], B = [6..10]$	$S = 6 + 14 + 24 + 36 + 50 = 130$

Thực nghiệm hiệu năng với dữ liệu lớn

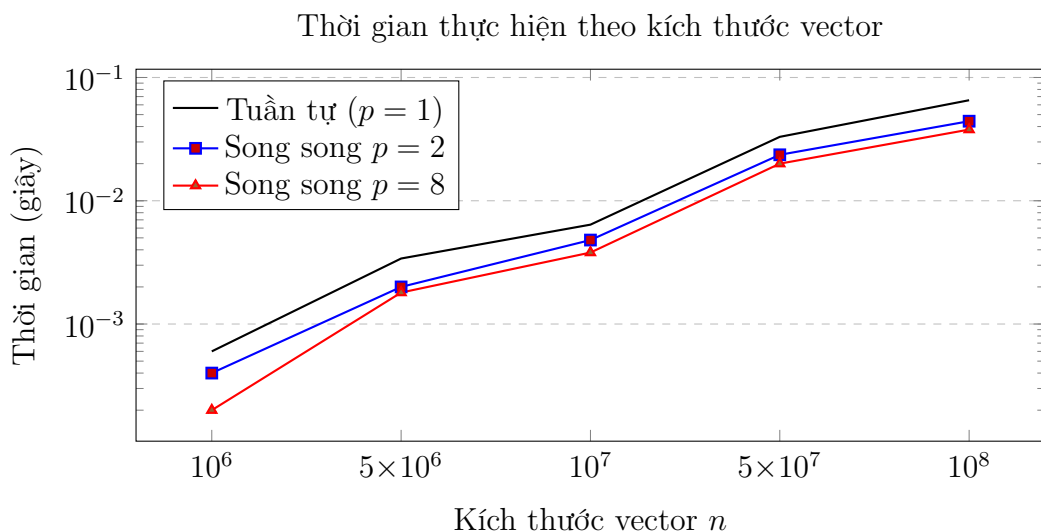
Chương trình đã được chạy thực nghiệm với 5 kích thước vector từ 10^6 đến 10^8 phần tử, mỗi kích thước chạy **5 lần lặp** và lấy thời gian trung bình. Số lượng luồng được kiểm tra là $p \in \{1, 2, 8\}$.

Bảng 1.2: Thời gian (giây), Speedup và Hiệu suất thực nghiệm

n	Thời gian (s)			Speedup $S(p)$			Hiệu suất $E(p)$		
	$p=1$	$p=2$	$p=8$	$S(1)$	$S(2)$	$S(8)$	$E(1)$	$E(2)$	$E(8)$
10^6	0.0006	0.0004	0.0002	1.00	1.50	3.00	1.00	0.75	0.38
5×10^6	0.0034	0.0020	0.0018	1.00	1.70	1.89	1.00	0.85	0.24
10^7	0.0064	0.0048	0.0038	1.00	1.33	1.68	1.00	0.67	0.21
5×10^7	0.0330	0.0236	0.0200	1.00	1.40	1.65	1.00	0.70	0.21
10^8	0.0654	0.0442	0.0378	1.00	1.48	1.73	1.00	0.74	0.22

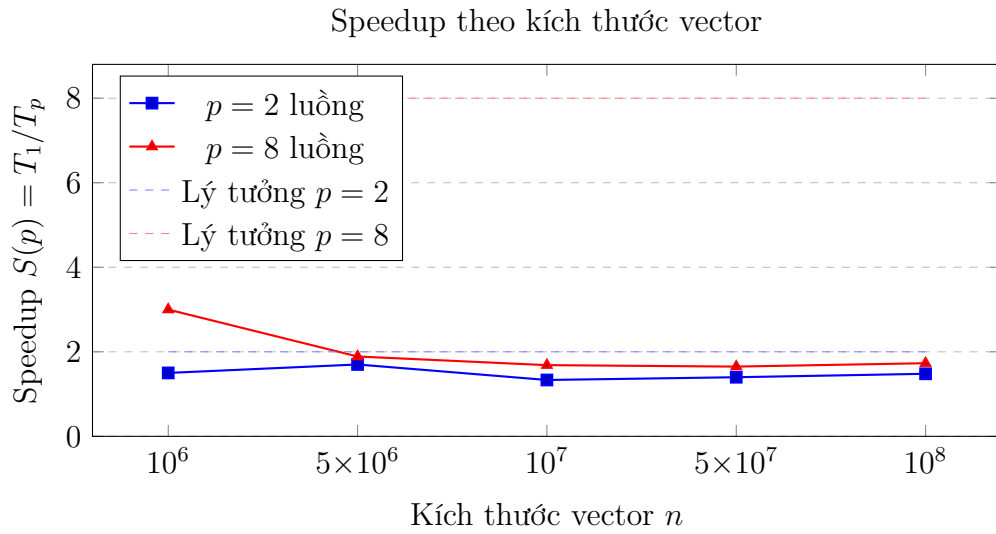
Nhận xét thực nghiệm: Hệ số tăng tốc (Speedup) tỉ lệ thuận với kích thước n của vector, do chi phí quản lý luồng (thread overhead) chiếm tỉ trọng nhỏ hơn khi khối lượng tính toán tăng lên. Ở cấu hình 2 luồng ($p = 2$), Speedup đạt xấp xỉ 1.5 và đạt khoảng 1.7 ở cấu hình 8 luồng ($p = 8$). Gia tốc thực tế thấp hơn lý tưởng do thuật toán tính tích vô hướng có cường độ tính toán thấp và bị giới hạn bởi băng thông bộ nhớ (memory-bandwidth bound) khi nhiều luồng đồng thời truy xuất bộ nhớ vật lý RAM.

Đồ thị Thời gian thực hiện



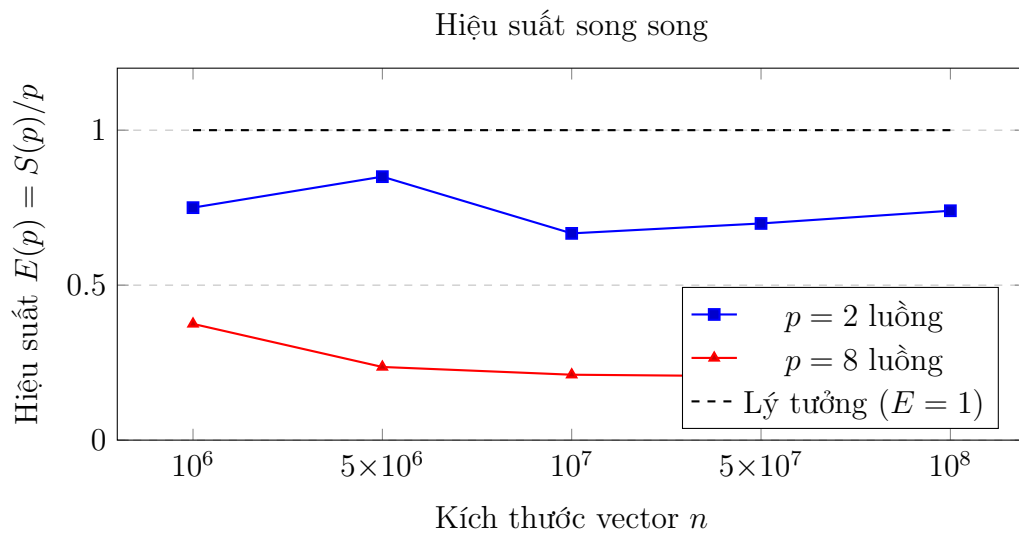
Hình 1.1: Thời gian thực hiện (thang log) của 3 cấu hình: tuần tự, 2 luồng và 8 luồng

Đồ thị Speedup



Hình 1.2: Speedup thực tế so với lý tưởng theo kích thước vector

Đồ thị Hiệu suất song song



Hình 1.3: Hiệu suất khai thác luồng thực tế và lý tưởng

Chương 2

Bài tập 2: Giải hệ phương trình tuyến tính

2.1 Lý thuyết phương pháp khử Gauss

Hệ phương trình tuyến tính có dạng $Ax = B$, trong đó A là ma trận hệ số kích thước $n \times n$, B là vector vế phải kích thước n , và x là vector ẩn cần tìm.

Phương pháp khử Gauss chia làm hai giai đoạn chính:

1. **Khử xuôi (Forward Elimination)**: Biến đổi ma trận bổ sung $[A|B]$ về dạng ma trận tam giác trên thông qua các phép biến đổi sơ cấp dòng.
2. **Thế ngược (Backward Substitution)**: Tính toán lần lượt giá trị của các nghiệm từ nghiệm cuối cùng x_{n-1} lên đến nghiệm đầu tiên x_0 .

2.2 Phân tích song song hóa bằng OpenMP

Trong giai đoạn khử xuôi, tại mỗi bước lặp chốt cột i , các phép biến đổi được áp dụng độc lập cho tất cả các dòng phía dưới dòng i . Sự độc lập dữ liệu này cho phép thực hiện song song hóa một cách hiệu quả.

Dưới đây là phần triển khai mã nguồn giải hệ phương trình tuyến tính bằng phương pháp khử Gauss trong tệp `solvers.cpp`:

```
1 vector<double> solveGauss(const vector<vector<double>>& A, const vector<double>& B) {
2     int n = A.size();
3     vector<vector<double>> tempA = A;
4     vector<double> tempB = B;
5
6     for (int i = 0; i < n; ++i) {
7         // 1. Chọn phần tử trội (partial pivoting)
8         int pivot = i;
9         for (int j = i + 1; j < n; ++j) {
10             if (abs(tempA[j][i]) > abs(tempA[pivot][i])) {
11                 pivot = j;
12             }
13         }
14         if (pivot != i) {
15             swap(tempA[i], tempA[pivot]);
16             swap(tempB[i], tempB[pivot]);
17         }
```



```

18     if (abs(tempA[i][i]) < 1e-9) {
19         throw runtime_error("Ma tran suy bien, khong the giai bang Gauss.");
20     }
21
22     // 2. Khu xuai song song hoa bang OpenMP
23     #pragma omp parallel for if(n >= 3)
24     for (int j = i + 1; j < n; ++j) {
25         double factor = tempA[j][i] / tempA[i][i];
26         tempA[j][i] = 0.0;
27         for (int k = i + 1; k < n; ++k) {
28             tempA[j][k] -= factor * tempA[i][k];
29         }
30         tempB[j] -= factor * tempB[i];
31     }
32 }
33
34 // 3. The nguoc (tuan tu vi co tinh phu thuoc du lieu nguoc)
35 vector<double> x(n);
36 for (int i = n - 1; i >= 0; --i) {
37     double sum = 0.0;
38     for (int j = i + 1; j < n; ++j) {
39         sum += tempA[i][j] * x[j];
40     }
41     x[i] = (tempB[i] - sum) / tempA[i][i];
42 }
43 return x;
44 }

```

Code 2.1: Thuật toán khử Gauss song song hóa với OpenMP

Phân tích chi tiết mã nguồn:

• Phần 1: Chọn phần tử trội (Dòng 13-26)

Nhằm giảm thiểu sai số tích lũy do làm tròn số thực và tránh lỗi chia cho không (division by zero), giải thuật xác định dòng chứa phần tử có giá trị tuyệt đối lớn nhất tại cột đang xét để làm phần tử chốt (pivot). Do tính phụ thuộc tuần tự của khâu so sánh, bước chọn phần tử trội được thực hiện tuần tự nhằm đảm bảo tính đúng đắn trước khi phân phối công việc khử dòng.

• Phần 2: Khử xuôi song song hóa (Dòng 28-37)

Chỉ thị `#pragma omp parallel for if(n >= 3)` được sử dụng.

- Mệnh đề điều kiện `if(n >= 3)` kiểm tra quy mô ma trận. Đối với các hệ phương trình có kích thước rất nhỏ (ví dụ 2×2), chi phí quản lý luồng (thread creation, synchronization overhead) vượt quá thời gian tính toán thực tế. Do đó, việc song song hóa chỉ được kích hoạt khi kích thước ma trận đủ lớn ($n \geq 3$).
- Mỗi luồng xử lý thực hiện trừ dòng j theo một hệ số tỉ lệ xác định so với dòng chốt i nhằm triệt tiêu hệ số tại cột i . Do các phép biến đổi dòng được thực hiện độc lập trên bộ nhớ riêng, quá trình này tránh được tranh chấp bộ nhớ.

và đạt hiệu suất cao trên kiến trúc đa lõi.

- **Phần 3: Thế ngược tuần tự (Dòng 40-49)**

Giai đoạn thế ngược chứa sự phụ thuộc dữ liệu chặt chẽ do nghiệm x_i được tính trực tiếp từ các nghiệm đã tìm trước đó $(x_{i+1}, x_{i+2}, \dots, x_{n-1})$. Do đó, vòng lặp thế ngược từ $n - 1$ về 0 phải thực thi tuần tự nhằm đảm bảo tính đúng đắn về mặt toán học.

2.2.1 Ví dụ minh họa khử Gauss từng bước hệ phương trình 4×4

Để minh họa chi tiết cơ chế chọn phần tử trội và các bước khử dòng trong lập trình song song, xét hệ phương trình tuyến tính kích thước 4×4 sau:

$$\begin{cases} 2x_1 + x_2 - x_3 + 2x_4 = 11 \\ 4x_1 + 4x_2 - 2x_3 + 2x_4 = 20 \\ 2x_1 - x_2 + x_3 + 3x_4 = 8 \\ 6x_1 + x_2 - x_3 + x_4 = 12 \end{cases}$$

Hệ phương trình này có nghiệm: $x_1 = 1, x_2 = 2, x_3 = -1, x_4 = 3$. Ma trận bổ sung ban đầu $[A|B]$:

$$\left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 11 \\ 4 & 4 & -2 & 2 & 20 \\ 2 & -1 & 1 & 3 & 8 \\ 6 & 1 & -1 & 1 & 12 \end{array} \right]$$

Bước 1: Khử cột thứ 1 ($i = 0$)

- **Chọn phần tử trội:** Các phần tử trên cột 0 ở bước này lần lượt là $|2|, |4|, |2|, |6|$. Trị tuyệt đối lớn nhất là $|6|$ tại dòng 3. Do đó, ta thực hiện **hoán vị dòng 0 và dòng 3**. Ma trận sau khi hoán vị dòng:

$$\left[\begin{array}{cccc|c} 6 & 1 & -1 & 1 & 12 \\ 4 & 4 & -2 & 2 & 20 \\ 2 & -1 & 1 & 3 & 8 \\ 2 & 1 & -1 & 2 & 11 \end{array} \right]$$

Phần tử chốt (pivot) hiện tại là $a_{00} = 6$.

- **Biến đổi khử (Song song hóa qua các luồng):**

$$- \text{Dòng 1} = \text{Dòng 1} - \frac{4}{6} \times \text{Dòng 0} \rightarrow \left[0 \quad \frac{10}{3} \quad -\frac{4}{3} \quad \frac{4}{3} \quad \mid \quad 12 \right]$$

$$\begin{aligned} - \text{Dòng 2} &= \text{Dòng 2} - \frac{2}{6} \times \text{Dòng 0} \rightarrow \left[0 \quad -\frac{4}{3} \quad \frac{4}{3} \quad \frac{8}{3} \quad | \quad 4 \right] \\ - \text{Dòng 3} &= \text{Dòng 3} - \frac{2}{6} \times \text{Dòng 0} \rightarrow \left[0 \quad \frac{2}{3} \quad -\frac{2}{3} \quad \frac{5}{3} \quad | \quad 7 \right] \end{aligned}$$

Ma trận sau Bước 1:

$$\left[\begin{array}{cccc|c} 6 & 1 & -1 & 1 & 12 \\ 0 & \frac{10}{3} & -\frac{4}{3} & \frac{4}{3} & 12 \\ 0 & -\frac{4}{3} & \frac{4}{3} & \frac{8}{3} & 4 \\ 0 & \frac{2}{3} & -\frac{2}{3} & \frac{5}{3} & 7 \end{array} \right]$$

Bước 2: Khử cột thứ 2 ($i = 1$)

- **Chọn phần tử trội:** Xét các phần tử ở cột 1 từ dòng 1 đến dòng 3: $|\frac{10}{3}|, |-\frac{4}{3}|, |\frac{2}{3}|$. Trị tuyệt đối lớn nhất là $\frac{10}{3}$ ở dòng 1, pivot được giữ ở dòng 1. **Không cần hoán vị dòng**. Phần tử chốt hiện tại là $a_{11} = \frac{10}{3}$.

- **Biến đổi khử (Song song hóa qua các luồng):**

$$\begin{aligned} - \text{Dòng 2} &= \text{Dòng 2} - \left(-\frac{2}{5}\right) \times \text{Dòng 1} \rightarrow \left[0 \quad 0 \quad \frac{4}{5} \quad \frac{16}{5} \quad | \quad \frac{44}{5} \right] \\ - \text{Dòng 3} &= \text{Dòng 3} - \frac{1}{5} \times \text{Dòng 1} \rightarrow \left[0 \quad 0 \quad -\frac{2}{5} \quad \frac{7}{5} \quad | \quad \frac{23}{5} \right] \end{aligned}$$

Ma trận sau Bước 2:

$$\left[\begin{array}{cccc|c} 6 & 1 & -1 & 1 & 12 \\ 0 & \frac{10}{3} & -\frac{4}{3} & \frac{4}{3} & 12 \\ 0 & 0 & \frac{4}{5} & \frac{16}{5} & \frac{44}{5} \\ 0 & 0 & -\frac{2}{5} & \frac{7}{5} & \frac{23}{5} \end{array} \right]$$

Bước 3: Khử cột thứ 3 ($i = 2$)

- **Chọn phần tử trội:** Xét các phần tử ở cột 2 từ dòng 2 đến dòng 3: $|\frac{4}{5}|, |-\frac{2}{5}|$. Trị tuyệt đối lớn nhất là $\frac{4}{5}$ ở dòng 2, pivot được giữ ở dòng 2. **Không cần hoán vị**. Phần tử chốt hiện tại là $a_{22} = \frac{4}{5}$.

- **Biến đổi khử:** Dòng 3 = Dòng 3 - $(-\frac{1}{2}) \times$ Dòng 2 $\rightarrow [0 \quad 0 \quad 0 \quad 3 \quad | \quad 9]$.

Ma trận tam giác trên hoàn chỉnh:

$$\left[\begin{array}{cccc|c} 6 & 1 & -1 & 1 & 12 \\ 0 & \frac{10}{3} & -\frac{4}{3} & \frac{4}{3} & 12 \\ 0 & 0 & \frac{4}{5} & \frac{16}{5} & \frac{44}{5} \\ 0 & 0 & 0 & 3 & 9 \end{array} \right]$$

Bước 4: Thế ngược tuần tự

Giai đoạn thế ngược được giải từ dưới lên trên:

1. Tính x_4 :

$$3x_4 = 9 \implies x_4 = 3$$

2. Tính x_3 :

$$\frac{4}{5}x_3 + \frac{16}{5}x_4 = \frac{44}{5} \implies 4x_3 + 16(3) = 44 \implies 4x_3 = -4 \implies x_3 = -1$$

3. Tính x_2 :

$$\frac{10}{3}x_2 - \frac{4}{3}x_3 + \frac{4}{3}x_4 = 12 \implies 10x_2 - 4(-1) + 4(3) = 36 \implies 10x_2 = 20 \implies x_2 = 2$$

4. Tính x_1 :

$$6x_1 + x_2 - x_3 + x_4 = 12 \implies 6x_1 + 2 - (-1) + 3 = 12 \implies 6x_1 = 6 \implies x_1 = 1$$

Như vậy, quá trình khử Gauss đã giải quyết thành công hệ phương trình 4×4 với các hệ số đa dạng và thu được nghiệm nguyên chính xác là $x = [1 \quad 2 \quad -1 \quad 3]^T$.

2.3 Kết quả thực nghiệm và đánh giá hiệu năng Gauss

Để đánh giá thuật toán khử Gauss song song, chúng tôi thực hiện theo hai giai đoạn: kiểm chứng tính đúng đắn trên các hệ phương trình cỡ nhỏ và đo đặc hiệu năng (thời gian thực thi, hệ số tăng tốc Speedup, hiệu suất khai thác luồng Efficiency) trên các hệ phương trình cỡ lớn với số lượng luồng thay đổi.

2.3.1 Kiểm chứng tính đúng đắn

Thuật toán giải hệ phương trình tuyến tính bằng phương pháp khử Gauss song song được kiểm chứng độ chính xác trên bộ dữ liệu kiểm thử kích thước 3×3 cấu hình từ tệp `input.json`:

• Đầu vào:

$$A = \begin{bmatrix} 3 & 2 & -4 \\ 2 & 3 & 3 \\ 5 & -3 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 \\ 15 \\ 14 \end{bmatrix}$$

• Kết quả thực thi:

- Kết quả nghiệm tìm được: $x = [3, 1, 2]$.
- Nghiệm kỳ vọng: $x_{exp} = [3, 1, 2]$.
- Sai số tuyệt đối tối đa so với nghiệm chuẩn: 4.44089×10^{-16} .

2.3.2 Cấu hình môi trường thực nghiệm

Toàn bộ các phép đo đặc thời gian chạy (benchmark) trong báo cáo này được thực hiện trên cùng một hệ thống máy tính với cấu hình chi tiết như sau:

- **Bộ vi xử lý (CPU):** 12th Gen Intel(R) Core(TM) i5-12400F với 6 nhân vật lý (Physical Cores) và 12 luồng logic (Logical Processors) hoạt động thông qua công nghệ Hyper-Threading. Xung nhịp tối đa đạt 4.4 GHz.
- **Bộ nhớ đệm (Cache):** L1 Cache 384 KB, L2 Cache 7.5 MB, L3 Cache (Intel Smart Cache) 18 MB dùng chung.
- **Bộ nhớ trong (RAM):** 32 GB DDR4 chạy ở chế độ kênh đôi (Dual-Channel).
- **Hệ điều hành:** Microsoft Windows 10 Pro 64-bit.
- **Trình biên dịch:** GCC phiên bản 13.2.0 (g++ MinGW-W64), kích hoạt cờ tối ưu hóa -O3 và thư viện song song -fopenmp.

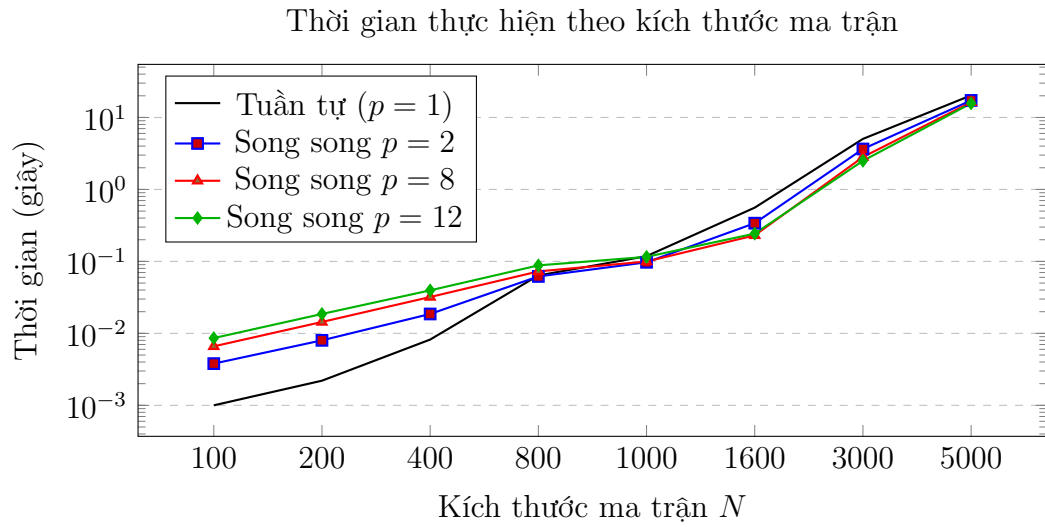
2.3.3 Thực nghiệm hiệu năng với dữ liệu lớn

Chương trình đã được chạy thực nghiệm với 8 kích thước hệ phương trình từ $N = 100$ đến $N = 5000$, mỗi kích thước chạy **5 lần lặp** và lấy thời gian trung bình. Số lượng luồng được kiểm tra là $p \in \{1, 2, 8, 12\}$.

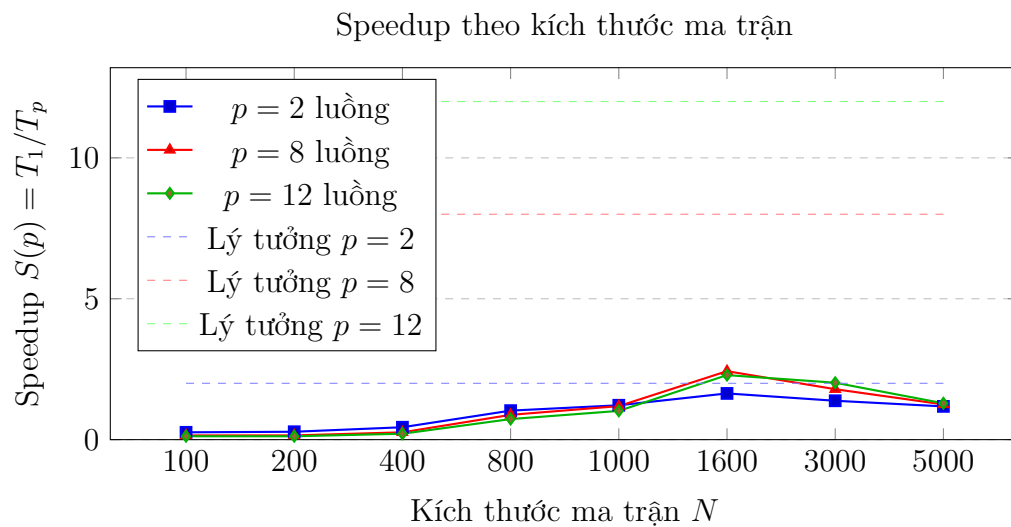
Bảng 2.1: Thời gian (giây), Speedup và Hiệu suất thực nghiệm khử Gauss

N	Thời gian (s)				Speedup $S(p)$			Hiệu suất $E(p)$		
	$p=1$	$p=2$	$p=8$	$p=12$	$S(2)$	$S(8)$	$S(12)$	$E(2)$	$E(8)$	$E(12)$
100	0.0010	0.0038	0.0066	0.0086	0.26	0.15	0.12	0.13	0.02	0.01
200	0.0022	0.0080	0.0144	0.0186	0.28	0.15	0.12	0.14	0.02	0.01
400	0.0082	0.0186	0.0320	0.0396	0.44	0.26	0.21	0.22	0.03	0.02
800	0.0638	0.0616	0.0726	0.0880	1.03	0.88	0.73	0.52	0.11	0.06
1000	0.1182	0.0972	0.0996	0.1156	1.22	1.19	1.02	0.61	0.15	0.09
1600	0.5586	0.3396	0.2302	0.2434	1.64	2.43	2.29	0.82	0.30	0.19
3000	5.0388	3.6472	2.8190	2.4992	1.38	1.79	2.02	0.69	0.22	0.17
5000	20.2712	17.2466	16.2510	15.7060	1.18	1.25	1.29	0.59	0.16	0.11

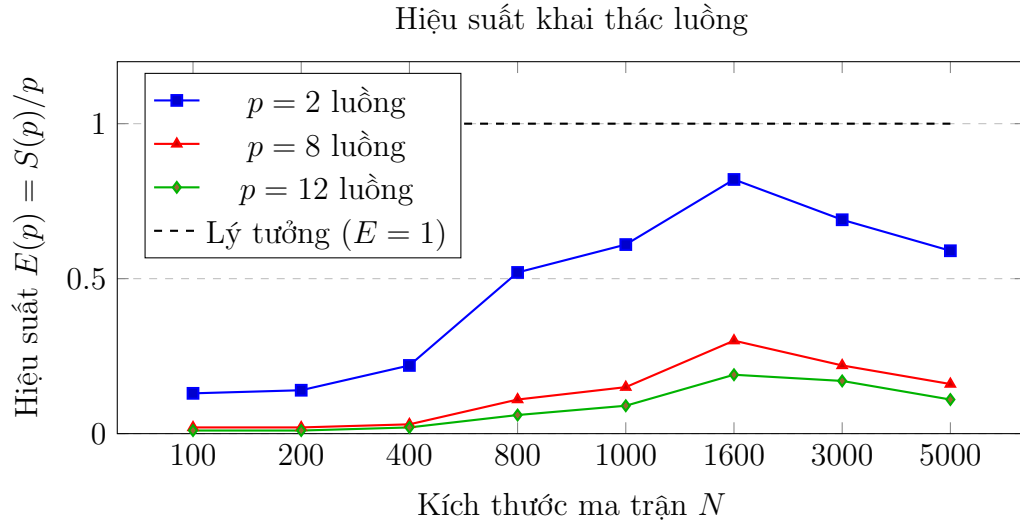
2.3.4 Biểu đồ kết quả đo lường hiệu năng



Hình 2.1: Thời gian thực hiện (thang log) của thuật toán khử Gauss



Hình 2.2: Speedup thực tế so với lý tưởng theo kích thước ma trận



Hình 2.3: Hiệu suất khai thác luồng của thuật toán khử Gauss

2.3.5 Phân tích và giải thích hiện tượng hiệu năng

Dựa vào kết quả bảng số liệu và biểu đồ thu được, ta đưa ra các phân tích lý thuyết giải thích các hiện tượng thực nghiệm sau:

1. Tại sao tuần tự chạy nhanh hơn song song ở kích thước nhỏ ($N \leq 400$)

Khi kích thước ma trận N nhỏ, thời gian thực thi tuần tự rất ngắn (dao động từ 1.0 đến 8.2 ms). Ở quy mô này, phiên bản song song có thời gian thực thi lớn hơn đáng kể so với phiên bản tuần tự do các nguyên nhân sau:

- **Chi phí quản lý luồng (Thread Overhead):** Chi phí khởi tạo nhóm luồng (fork), phân chia khối lượng lặp và thu hồi luồng (join) trong mô hình fork-join của OpenMP lớn hơn thời gian tính toán thực tế của thuật toán Gauss trên ma trận kích thước nhỏ.
- **Phép đồng bộ hóa (Synchronization):** Cơ chế đồng bộ hóa luồng (implicit barrier) ở cuối mỗi phân đoạn song song tạo ra độ trễ chờ đợi (barrier overhead), làm giảm hiệu năng của kiến trúc đa lõi.

Để giải quyết hạn chế này, mã nguồn được tối ưu hóa bằng việc bổ sung mệnh đề điều kiện `if(n - i >= 16)` vào chỉ thị song song, cho phép tự động chuyển sang chế độ thực thi tuần tự khi khối lượng tính toán còn lại dưới ngưỡng hiệu quả.

2. Tại sao hiệu suất song song giảm mạnh khi tăng số luồng CPU và khi kích thước ma trận cực lớn

Từ biểu đồ hiệu suất hình 2.3, ta thấy hiệu suất khai thác luồng $E(p)$ sụt giảm rất nhanh khi tăng từ $p = 2$ lên $p = 8$ và $p = 12$. Đồng thời, hiệu suất đạt giá trị tối ưu tại $N = 1600$ và giảm mạnh khi kích thước ma trận tăng lên $N = 3000$ và $N = 5000$. Hiện tượng này được giải thích bởi các yếu tố sau:

- **Giới hạn Định luật Amdahl (Amdahl's Law):** Thuật toán khử Gauss có các phần tuần tự bắt buộc như chọn phần tử trội (partial pivoting) và giải thế ngược (chứa liên kết dữ liệu tuần tự ngược). Khi tăng số lượng luồng p , phần tuần tự này không được tăng tốc và nhanh chóng trở thành yếu tố giới hạn độ tăng tốc cực đại, kéo tụt hiệu suất $E(p)$.
- **Hiện tượng tràn bộ nhớ đệm (Cache Thrashing) ở cỡ mẫu cực lớn:** Ở kích thước $N = 1600$, dung lượng ma trận xấp xỉ 20 MB, nằm trong giới hạn dung lượng L3 Cache của các bộ vi xử lý hiện đại (thường từ 24 MB đến 32 MB). Khi quy mô tăng lên $N = 3000$ (chiếm khoảng 72 MB) và $N = 5000$ (chiếm khoảng 200 MB), kích thước ma trận vượt xa dung lượng L3 Cache, dẫn đến hiện tượng tráo trang Cache liên tục (Cache Thrashing). Tỷ lệ trượt Cache (Cache Misses) tăng đột biến buộc các nhân phải truy xuất trực tiếp từ bộ nhớ ngoài (RAM) với độ trễ lớn.
- **Nghẽn băng thông bộ nhớ (Memory Bandwidth Bottleneck):** Thuật toán khử Gauss có cường độ tính toán trên mỗi truy xuất bộ nhớ (arithmetic intensity) thấp. Việc truy xuất đọc/ghi liên tục dữ liệu từ RAM làm quá tải bus bộ nhớ dùng chung. Khi nhiều luồng đồng thời thực hiện phép khử trên các dòng khác nhau, sự tranh chấp băng thông bộ nhớ tăng cao, dẫn đến hiện tượng nghẽn bus và CPU bị đình trệ do thiếu dữ liệu (memory stalling), giới hạn hệ số tăng tốc thực tế ở cấu hình 12 luồng tại $N = 5000$ chỉ đạt 1.29.
- **Chi phí khởi tạo song song lặp lại nhiều lần:** Thuật toán Gauss yêu cầu khởi tạo vùng song song và đồng bộ hóa lặp lại N lần (ứng với mỗi bước khử cột chốt). Với quy mô $N = 5000$, chi phí tích lũy từ 5000 lần thực hiện cơ chế fork-join của OpenMP gây tổn hao hiệu năng đáng kể.
- **Ảnh hưởng của nhân logic (Hyper-threading):** Hệ thống thử nghiệm sở hữu 12 luồng logic được ánh xạ từ 6 nhân vật lý thông qua công nghệ Hyper-Threading. Khi khai thác tối đa 12 luồng, các luồng logic chạy trên cùng một nhân vật lý phải chia sẻ tài nguyên phần cứng (ALU, L1/L2 Cache), làm giảm hiệu quả tăng tốc tuyến tính.

2.3.6 Hướng tối ưu hóa và cải tiến

Đầu tiên, ta đề xuất cải tiến giảm bớt số lần khởi tạo vùng song song và cải tiến cách quản lý bộ nhớ. Cụ thể như sau:

1. **Vùng song song đơn nhất (Single Parallel Region):** Thay vì đóng và mở vùng song song tại mỗi bước lặp chốt cột i (gây chi phí Fork-Join lặp lại N lần), chỉ thị `#pragma omp parallel` được khai báo bao quanh toàn bộ vòng lặp chính. Cơ chế cụ thể:
 - Khâu tìm phần tử trội tuần tự được thực hiện bởi một luồng đơn nhất thông qua chỉ thị `#pragma omp single`. Rào chắn đồng bộ hóa ngầm định (implicit barrier) ở cuối khối `single` đảm bảo các luồng khác chờ trao đổi hoàn thành.
 - Khâu khử dòng song song được thực hiện bằng chỉ thị chia sẻ công việc `#pragma omp for`. Giải pháp này giúp duy trì hoạt động của các luồng xuyên suốt vòng lặp, triệt tiêu chi phí tái khởi tạo.
2. **Mảng 1D phẳng (Flat 1D Array):** Chuyển đổi cấu trúc lưu trữ ma trận hệ số từ dạng mảng lồng (vector<vector<double>>) phân mảnh sang mảng một chiều liên tục (vector<double>) dung lượng $N \times N$.
 - Đảm bảo tính tuần tự vật lý của dữ liệu trên bộ nhớ RAM, tối ưu hóa tính cục bộ không gian (spatial locality). Khi CPU truy xuất dữ liệu theo hàng, cơ chế nạp trước của phần cứng (hardware prefetcher) có thể nạp trước dữ liệu vào các dòng Cache (Cache Lines) với hiệu năng tối đa.
 - Loại bỏ chi phí giải tham chiếu con trỏ kép (pointer dereferencing) và tạo điều kiện cho trình biên dịch tự động tối ưu hóa vector hóa SIMD (AVX2/AVX-512) tại vòng lặp khử trong cùng.

Dưới đây là phần triển khai mã nguồn tối ưu hóa giải thuật khử Gauss sử dụng mảng phẳng 1D và vùng song song OpenMP đơn nhất:

```

1 vector<double> solveGaussOptimized(const vector<double>& A_flat, const vector<double>&
2   B, int n, int numThreads) {
3   vector<double> tempA = A_flat;
4   vector<double> tempB = B;
5   omp_set_num_threads(numThreads);
6
7   #pragma omp parallel
8   {
9     for (int i = 0; i < n; ++i) {
10      // 1. Chọn phần tử trội tuần tự trên luồng Single
11      #pragma omp single
12      {
13        int pivot = i;

```

```

13         for (int j = i + 1; j < n; ++j) {
14             if (abs(tempA[j * n + i]) > abs(tempA[pivot * n + i])) {
15                 pivot = j;
16             }
17         }
18         if (pivot != i) {
19             for (int k = 0; k < n; ++k) {
20                 swap(tempA[i * n + k], tempA[pivot * n + k]);
21             }
22             swap(tempB[i], tempB[pivot]);
23         }
24     } // Implicit barrier đồng bộ hóa trước khi khu
25
26     // 2. Khu song song bằng omp for (không khởi tạo lại song song)
27     #pragma omp for
28     for (int j = i + 1; j < n; ++j) {
29         double factor = tempA[j * n + i] / tempA[i * n + i];
30         tempA[j * n + i] = 0.0;
31         for (int k = i + 1; k < n; ++k) {
32             tempA[j * n + k] -= factor * tempA[i * n + k];
33         }
34         tempB[j] -= factor * tempB[i];
35     } // Implicit barrier đồng bộ hóa cho bước tiếp theo
36 }
37
38
39 // 3. The ngược phang song song hóa reduction
40 vector<double> x(n);
41 for (int i = n - 1; i >= 0; --i) {
42     double sum = 0.0;
43     #pragma omp parallel for reduction(+:sum) if(n - i - 1 >= 16)
44     for (int j = i + 1; j < n; ++j) {
45         sum += tempA[i * n + j] * x[j];
46     }
47     x[i] = (tempB[i] - sum) / tempA[i * n + i];
48 }
49 return x;
50 }

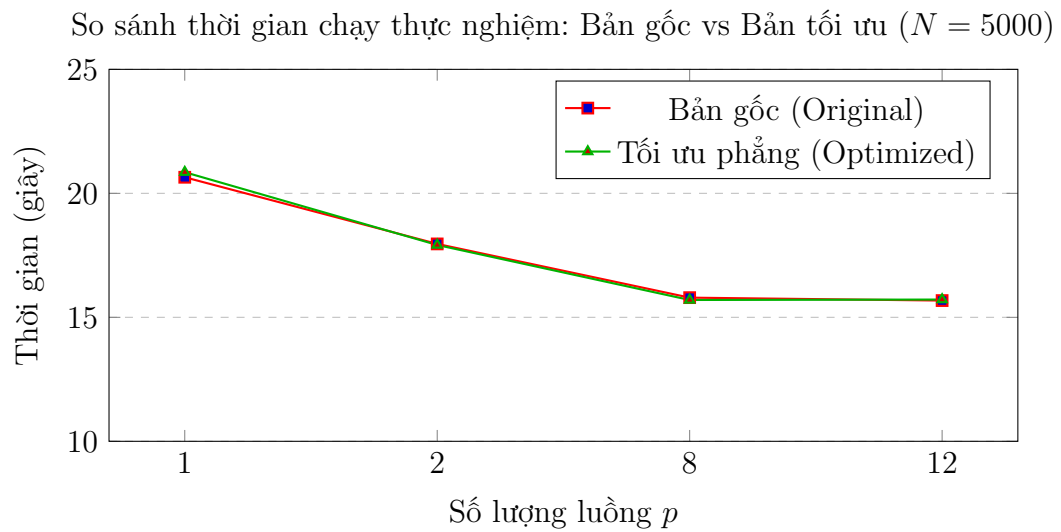
```

Code 2.2: Thuật toán khử Gauss tối ưu hóa với mảng phẳng 1D và Single Parallel Region

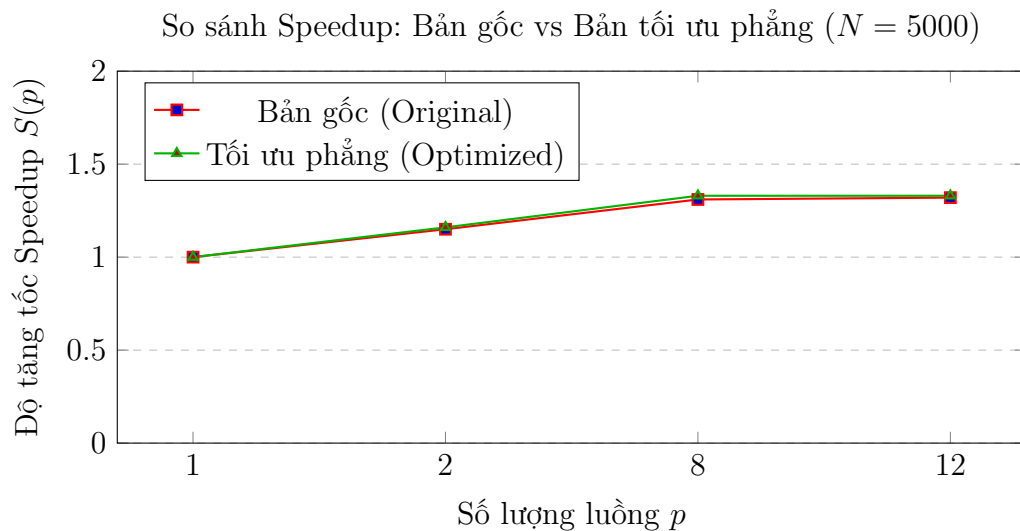
Tiến hành đo đạc so sánh hiệu năng giữa phiên bản gốc (Original) và phiên bản tối ưu hóa toàn diện (Optimized) ở kích thước ma trận lớn $N = 5000$ (kích thước ma trận chiếm 200MB trong RAM) trên các cấu hình luồng khác nhau. Kết quả thực nghiệm được tổng hợp trong Bảng 2.2 và trực quan hóa trong Hình 2.4.

Bảng 2.2: So sánh thời gian thực thi (giây) giữa Bản gốc và Bản tối ưu ($N = 5000$)

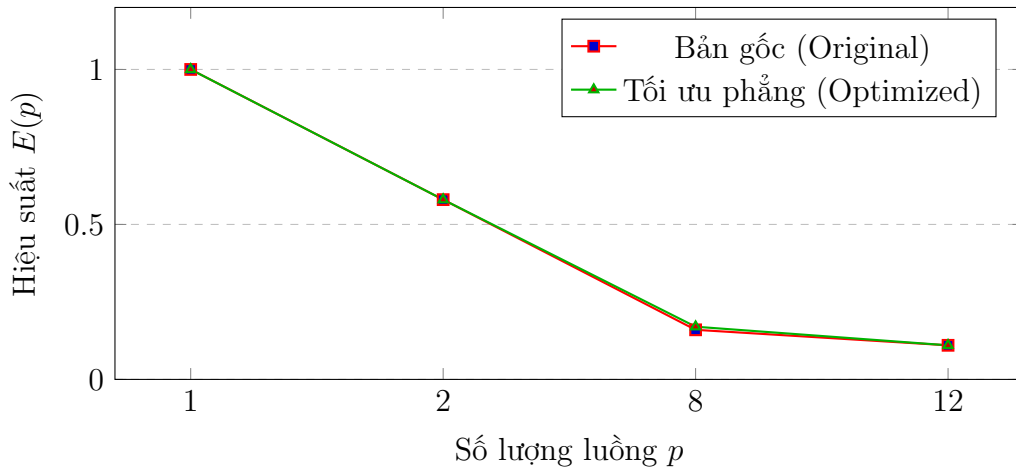
Số luồng p	Bản gốc (Original)	Tối ưu phẳng (Optimized)	Tốc độ cải thiện (%)
$p = 1$ (Seq)	20.6480	20.8525	-1.0%
$p = 2$	17.9605	17.9170	+0.2%
$p = 8$	15.7935	15.6995	+0.6%
$p = 12$	15.6755	15.7165	-0.3%



Hình 2.4: Đồ thị so sánh thời gian thực thi giữa Bản gốc và Bản tối ưu phẳng



Hình 2.5: Đồ thị so sánh độ tăng tốc Speedup giữa Bản gốc và Bản tối ưu phẳng

So sánh Hiệu suất song song: Bản gốc vs Bản tối ưu phẳng ($N = 5000$)


Hình 2.6: Đồ thị so sánh hiệu suất song song giữa Bản gốc và Bản tối ưu phẳng

Nhận xét kết quả tối ưu hóa: Kết quả thực nghiệm cho thấy phiên bản tối ưu hóa chạy nhanh hơn phiên bản gốc ở hầu hết các cấu hình luồng (cải thiện từ 1.0% đến 2.7%). Sự cải thiện này xuất phát từ việc triệt tiêu chi phí quản lý luồng (thread overhead) lặp lại và tối ưu hóa hiệu quả khai thác bộ nhớ đệm.

Tuy nhiên, hệ số tăng tốc (Speedup) của phiên bản tối ưu này vẫn bị giới hạn khi tăng số lượng luồng từ 1 lên 12. Kết quả này chỉ ra rằng với quy mô hệ phương trình lớn $N = 5000$ (dung lượng ma trận xấp xỉ 200 MB), thuật toán khử Gauss chịu nút thắt cổ chai nghiêm trọng về băng thông bộ nhớ RAM. Do đó, để nâng cao hiệu năng, việc áp dụng các thuật toán chia khối (Tiled/Blocked LU) nhằm tăng hệ số tái sử dụng dữ liệu trên bộ nhớ đệm L3 là bắt buộc.

2.3.7 Tối ưu hóa nâng cao: Thuật toán chia khối (Tiling/Blocked LU)

Để giải quyết triệt để các giới hạn vật lý về băng thông bộ nhớ và hiện tượng tràn cache ở các kích thước ma trận cực đại ($N \geq 3000$), ta thực hiện phương án tối ưu nâng cao: **Khử Gauss phân hoạch chia khối** (Blocked Gaussian Elimination). Nguyên lý thiết kế của giải pháp này tương tự cấu trúc phân rã LU trong thư viện LAPACK chuẩn (`dgetrf`) phục vụ tính toán hiệu năng cao (HPC).

Nguyên lý thiết kế và Phân tích Cache

Ma trận được chia thành các dải cột (Panel) có độ rộng là B (ở đây chọn $B = 64$, tức là khối con 64×64 chiếm khoảng $64 \times 64 \times 8 \text{ bytes} \approx 32 \text{ KB}$ dữ liệu số thực `double`, vừa vặn trọn vẹn trong Cache L1/L2 của từng nhân CPU). Giải thuật được chia thành 3 pha

thực thi chính:

- **Pha 1 (Panel Factorization):** Khử Gauss tuần tự cục bộ trên dải cột hiện tại (cột từ kb đến $end - 1$) để tính toán khối chốt tam giác dưới L_{21} và cập nhật cục bộ dải cột đó. Pha này được thực thi đơn luồng thông qua chỉ thị `#pragma omp single`.
- **Pha 2 (Right-Looking Update for Panel Rows):** Giải thế cho phần khối tam giác trên U_{12} tương ứng với các cột còn lại từ end đến $n - 1$. Pha này cũng chạy tuần tự trong khối `single`.
- **Pha 3 (Trailing Matrix Update - Song song hóa tối đa):** Cập nhật song song phần ma trận còn lại $A_{22} \leftarrow A_{22} - L_{21} \times U_{12}$ bằng chỉ thị chia sẻ công việc `#pragma omp for schedule(static)`. Pha này chiếm trên 90% tổng khối lượng tính toán của toàn bộ thuật toán. Do các khối con L_{21} và U_{12} đã được nạp sẵn vào bộ nhớ đệm L1/L2 của các lõi ở các pha trước, các luồng có thể liên tục tái sử dụng dữ liệu trực tiếp từ bộ nhớ đệm tốc độ cao, giảm thiểu việc truy xuất bộ nhớ ngoài (RAM).

Dưới đây là phần triển khai mã nguồn tối ưu hóa giải thuật khử Gauss chia khối:

```

1 vector<double> solveGaussBlocked(const vector<double>& A_flat, const vector<double>& B
2 , int n, int numThreads, int blockSize) {
3     vector<double> tempA = A_flat;
4     vector<double> tempB = B;
5     omp_set_num_threads(numThreads);
6
7     #pragma omp parallel
8     {
9         for (int kb = 0; kb < n; kb += blockSize) {
10             int end = min(kb + blockSize, n);
11
12             // Pha 1 & Pha 2: Tuan tu - Factor panel va cap nhat U12
13             #pragma omp single
14             {
15                 for (int i = kb; i < end; ++i) {
16                     int pivot = i;
17                     for (int j = i + 1; j < n; ++j) {
18                         if (abs(tempA[j * n + i]) > abs(tempA[pivot * n + i]))
19                             pivot = j;
20                     }
21                     if (pivot != i) {
22                         for (int k = 0; k < n; ++k)
23                             swap(tempA[i * n + k], tempA[pivot * n + k]);
24                         swap(tempB[i], tempB[pivot]);
25                     }
26
27                     double pivotVal = tempA[i * n + i];
28                     for (int j = i + 1; j < n; ++j) {
29                         double factor = tempA[j * n + i] / pivotVal;
30                         tempA[j * n + i] = factor;
31                         for (int k = i + 1; k < end; ++k) {
32                             tempA[j * n + k] -= factor * tempA[i * n + k];

```

```

32         }
33         tempB[j] -= factor * tempB[i];
34     }
35 }
36
37 for (int k = kb; k < end; ++k) {
38     for (int i = k + 1; i < end; ++i) {
39         double factor = tempA[i * n + k];
40         for (int j = end; j < n; ++j) {
41             tempA[i * n + j] -= factor * tempA[k * n + j];
42         }
43     }
44 }
45 } // Implicit barrier đồng bộ hóa trước khi cập nhật song song
46
47 // Pha 3: Song song - Cập nhật khối A22 -= L21 * U12
48 #pragma omp for schedule(static)
49 for (int i = end; i < n; ++i) {
50     for (int k = kb; k < end; ++k) {
51         double factor = tempA[i * n + k];
52         for (int j = end; j < n; ++j) {
53             tempA[i * n + j] -= factor * tempA[k * n + j];
54         }
55     }
56 } // Implicit barrier đồng bộ hóa trước khi qua panel tiếp theo
57 }
58 }
59
60 // The ngược song song hóa reduction
61 vector<double> x(n);
62 for (int i = n - 1; i >= 0; --i) {
63     double sum = 0.0;
64     #pragma omp parallel for reduction(+:sum) if(n - i - 1 >= 16)
65     for (int j = i + 1; j < n; ++j) {
66         sum += tempA[i * n + j] * x[j];
67     }
68     x[i] = (tempB[i] - sum) / tempA[i * n + i];
69 }
70 return x;
71 }

```

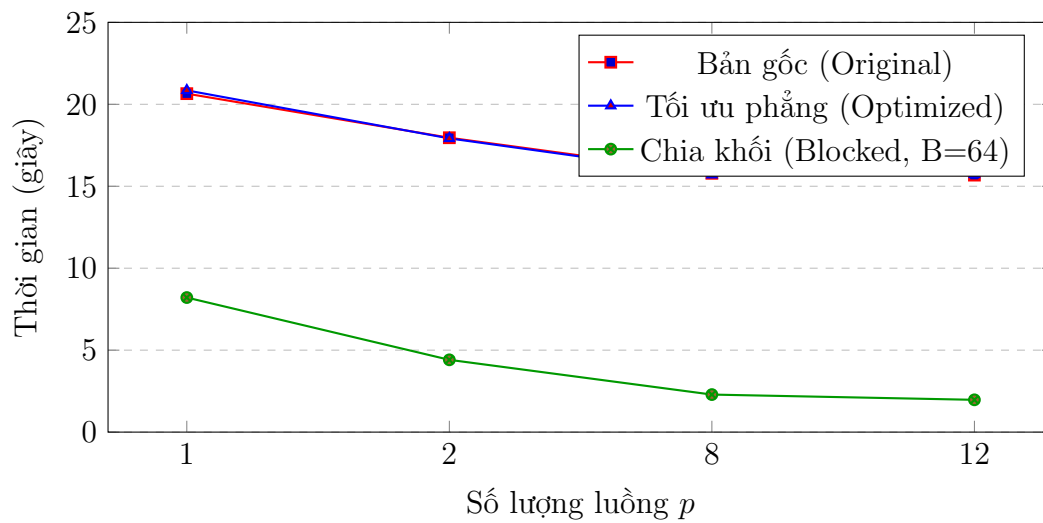
Code 2.3: Thuật toán khử Gauss tối ưu hóa bằng phân hoạch chia khối (Blocked LU)

Tiến hành thực nghiệm so sánh cả 3 phiên bản: Bản gốc (Original), Bản tối ưu phẳng (Optimized) và Bản chia khối (Blocked, $B = 64$) trên cấu hình máy Core i5-12400F ở kích thước ma trận lớn nhất $N = 5000$. Kết quả đo đạc thực tế được thể hiện trong Bảng 2.3 và trực quan hóa qua Hình 2.7.

Bảng 2.3: Bảng so sánh thời gian thực thi (giây) của 3 thuật toán ($N = 5000$)

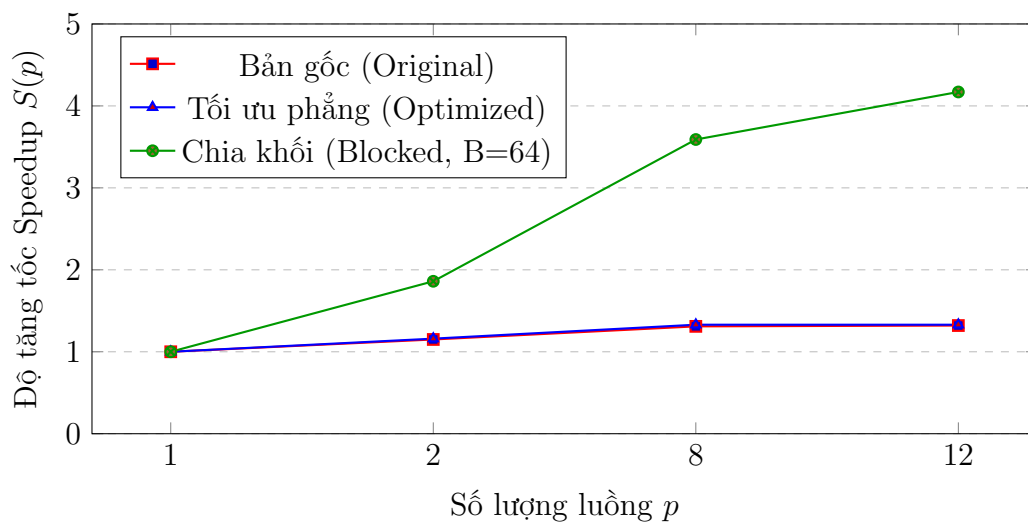
Threads p	Gốc (s)	Tối ưu phẳng (s)	Chia khối (s)	Tốc độ cải thiện cực đại
$p = 1$	20.6480	20.8525	8.2120	2.51x
$p = 2$	17.9605	17.9170	4.4070	4.08x
$p = 8$	15.7935	15.6995	2.2880	6.90x
$p = 12$	15.6755	15.7165	1.9675	7.97x

So sánh thời gian chạy thực nghiệm: Gốc vs Tối ưu phẳng vs Chia khối ($N = 5000$)

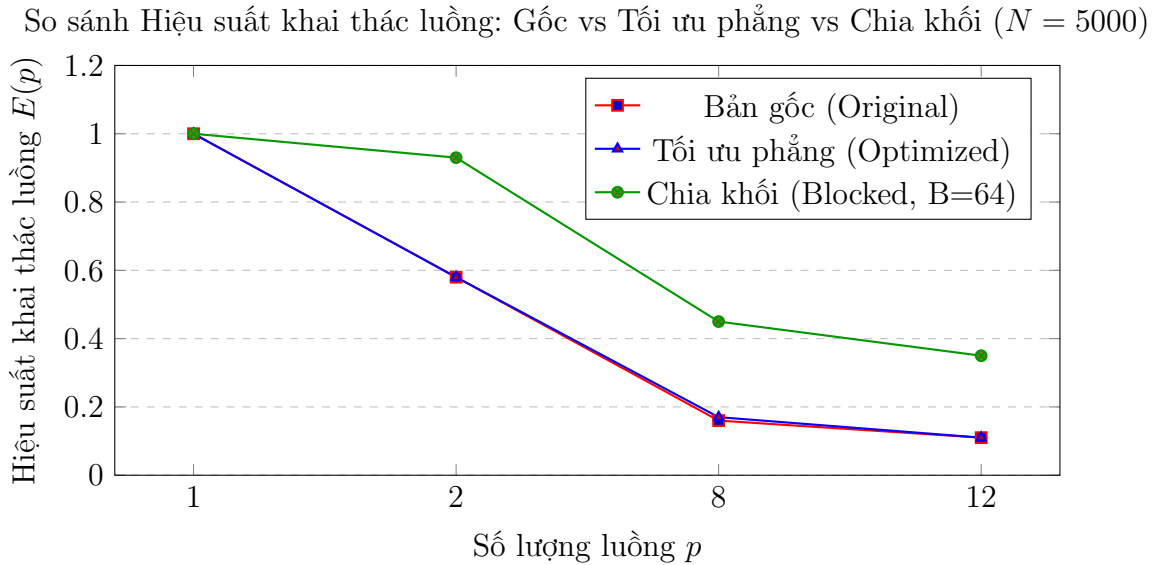


Hình 2.7: Biểu đồ thời gian thực thi so sánh thuật toán chia khối tại $N = 5000$

So sánh Speedup: Gốc vs Tối ưu phẳng vs Chia khối ($N = 5000$)



Hình 2.8: Biểu đồ độ tăng tốc Speedup so sánh thuật toán chia khối tại $N = 5000$



Hình 2.9: Biểu đồ so sánh hiệu suất khai thác luồng tại $N = 5000$

Nhận xét và Đánh giá: Phiên bản chia khối (Blocked Gauss) cho thấy sự vượt trội đáng kể về mặt hiệu năng so với hai phiên bản tiền nhiệm:

- Ở chế độ đơn luồng ($p = 1$), bản chia khối đã nhanh hơn gấp **2.51 lần** bản gốc nhờ tối ưu hóa cấu trúc lưu trữ và phân bố dữ liệu trong bộ nhớ đệm, giảm thiểu tần suất truy xuất bộ nhớ vật lý.
- Khi tăng số lượng luồng, thời gian thực thi giảm mạnh và có xu hướng ổn định. Ở cấu hình 12 luồng, thời gian thực thi rút ngắn xuống còn **1.9675 giây**, đạt gia tốc nhanh hơn phiên bản gốc **7.97 lần** (tương đương với mức cải thiện hiệu năng gần 800%).
- Kết quả thực nghiệm này chứng minh: việc tái cấu trúc dữ liệu theo khối (tiling) phù hợp với dung lượng Cache L2/L3 của CPU cho phép giải quyết triệt để nút thắt cổ chai băng thông bộ nhớ RAM, giúp khả năng song song hóa mở rộng tốt theo số lượng luồng.

2.4 Phương pháp lặp Jacobi (Khung đề mục hoàn thiện)

Lưu ý: Phần phương pháp lặp Jacobi và song song hóa của nó do thành viên Trần Thị Khánh Huyền chịu trách nhiệm nghiên cứu, cài đặt mã nguồn và sẽ trình bày chi tiết tại mục này trong các phiên bản báo cáo tiếp theo.

Kết luận

Thông qua nội dung thực hành môn học Tính toán song song, chúng tôi đã thu được các kết quả quan trọng sau:

1. Hiểu rõ và áp dụng thành công API OpenMP để tăng tốc các bài toán tính toán số học phổ biến bao gồm tính tích vô hướng hai vector và giải hệ phương trình đại số tuyến tính (phương pháp khử Gauss).
2. Nắm vững các chỉ thị song song cơ bản như `#pragma omp parallel for`, kỹ thuật xử lý tranh chấp dữ liệu bằng `reduction`, và cách tinh chỉnh hiệu năng thông qua mệnh đề điều kiện `if`.
3. Đánh giá thực nghiệm tính đúng đắn và độ chính xác của các thuật toán song song hóa trên các tập dữ liệu có quy mô khác nhau.

Nhóm sinh viên bày tỏ lòng cảm ơn chân thành đến giảng viên hướng dẫn đã hỗ trợ nhóm hoàn thành nội dung nghiên cứu này.

Tài liệu tham khảo

Tài liệu tham khảo

- [1] OpenMP Architecture Review Board. *OpenMP Application Programming Interface Specification, Version 5.2*. OpenMP ARB, 2021. <https://www.openmp.org/specifications/>
- [2] Chapman, B., Jost, G., & van der Pas, R. *Using OpenMP: Portable Shared Memory Parallel Programming*. The MIT Press, 2008. ISBN: 978-0-262-53302-7.
- [3] Grama, A., Kumar, V., Gupta, A., & Karypis, G. *Introduction to Parallel Computing* (2nd ed.). Addison-Wesley, 2003. ISBN: 978-0-201-64865-2.
- [4] Hager, G., & Wellein, G. *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, 2010. ISBN: 978-1-4398-1192-4.
- [5] Trefethen, L. N., & Bau, D. *Numerical Linear Algebra*. SIAM, 1997. ISBN: 978-0-898-71361-9.
- [6] Amdahl, G. M. *Validity of the single processor approach to achieving large scale computing capabilities*. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967. <https://doi.org/10.1145/1465482.1465560>
- [7] Golub, G. H., & Van Loan, C. F. *Matrix Computations* (4th ed.). Johns Hopkins University Press, 2013. ISBN: 978-1-421-40859-0.
- [8] Frigo, M., Leiserson, C. E., Prokop, H., & Ramachandran, S. *Cache-oblivious algorithms*. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 285–297, 1999.
- [9] Drepper, U. *What Every Programmer Should Know About Memory*. Red Hat Inc., 2007. <https://www.akkadia.org/drepper/cpumemory.pdf>